Final Report

Title

Stock Market Trend Analysis and Prediction Using Financial Data

Team Members

- Ryan Amir
- Gian Cases

Submission Links

- GitHub Repository: https://github.com/ryana79/cs210-stock-project
- Demo Video: https://youtu.be/s8H6e7xGIFA

1. Problem Definition and Background

Financial markets generate massive amounts of data every day, including stock prices, trading volumes, and other indicators that investors use to understand market behavior. Because of the size and complexity of financial datasets, it can be difficult to manually identify patterns or trends that influence stock price movements. Data science techniques provide a way to organize, analyze, and interpret this information more effectively.

This project investigates whether historical stock market data and derived financial indicators can be used to predict short-term stock price direction. The central research question is: *Can historical stock market data and financial indicators be used to predict short-term stock price movements using machine learning models?*

The project combines three major course components in one workflow: relational database design, data cleaning and visualization, and machine learning. Financial data is a strong application area for data management because it is time-based, high volume, and often reused across multiple stages of analysis.

2. Data Description

Data Source

The dataset was downloaded from Yahoo Finance (https://finance.yahoo.com) using the `yfinance` Python library. Yahoo Finance is a widely used, publicly available source of historical stock market data. The data is free to access and does not require authentication.

Selected Stocks

Three widely traded companies were selected for analysis:

Ticker	Company	Sector
AAPL	Apple Inc.	Technology
MSFT	Microsoft Corporation	Technology
TSLA	Tesla, Inc.	Automotive / Technology

These companies were chosen because they are among the most actively traded stocks on major exchanges, which ensures sufficient daily data and trading volume for meaningful analysis. Selecting stocks from overlapping but distinct sectors also allows for cross-sector comparison in the database layer.

Dataset Size

The cleaned dataset covers the date range from `2018-01-02` through the most recent available trading day. As of the latest pipeline run, the dataset contains `6,270` total daily rows across the three stocks (`2,090` trading days per stock over approximately 8 years). These counts increase as new trading days are added by Yahoo Finance.

Raw Features

Each daily record from Yahoo Finance contains the following fields:

Field	Type	Description
date	date	Trading day
open	float	Opening stock price
high	float	Highest price during the day
low	float	Lowest price during the day
close	float	Closing stock price
adj_close	float	Adjusted closing price (accounts for splits and dividends)
volume	integer	Number of shares traded

Cleaned Dataset

After cleaning, two additional fields are added:

Field	Type	Description
ticker	string	Stock ticker symbol (AAPL, MSFT, or TSLA)

company_name	string	Full company name

The full cleaned-data schema is documented in `docs/data_dictionary.md`.

3. Database Design

The project stores stock data in a relational SQLite database with two tables: `stocks` and `daily_prices`.

The `stocks` table contains one row per ticker and stores the company name and sector. Storing sector metadata in this table demonstrates normalization: rather than repeating the sector value across thousands of daily price rows, we store it once per stock and join when needed.

The `daily_prices` table stores one row per trading day for each stock, including open, high, low, close, adjusted close, and volume. Each price row references a stock through `stock_id`.

Key constraints:

- `ticker` is UNIQUE in `stocks` — prevents duplicate stock entries
- `(stock_id, trade_date)` is UNIQUE in `daily_prices` — prevents duplicate daily records
- `stock_id` is a FOREIGN KEY referencing `stocks` — enforces referential integrity
- An INDEX on `(stock_id, trade_date)` speeds up date-range queries

This design reduces repeated company metadata and keeps the design normalized. Separating stock metadata from daily records also makes SQL queries cleaner and supports efficient analysis across multiple stocks and time periods.

After loading the database, the pipeline automatically verifies that the number of rows in the database matches the cleaned CSV to ensure data integrity.

SQL Analysis

The project includes 11 documented SQL queries organized into two tiers:

Basic aggregate queries (7): row counts per stock, latest close prices, average close/volume per stock, yearly average close, price range per stock, date coverage per stock, and sector-level performance summary.

Advanced queries (4): top-10 daily gains and top-10 daily losses (using correlated subqueries to find the previous trading day's close), quarterly average close/volume (using CASE expressions for date-based bucketing), and sector performance summary (joining the sector metadata from the `stocks` table).

These queries demonstrate SQL capabilities ranging from basic GROUP BY aggregation to correlated subqueries and conditional expressions.

4. Data Cleaning and Preparation

The raw Yahoo Finance CSV files were cleaned in a repeatable Python pipeline (`src/data/`

`clean_data.py`) that applies **eight explicit data quality rules**, organized into hard checks (rows that fail are dropped) and soft checks (rows that fail are flagged but kept). Every rule writes a row to `data/processed/data_validation_report.csv`, so the cleaning behaviour is auditable even when no rows are removed.

Hard checks (drop rows that fail)

#	Rule	Rationale
1	`schema_required_columns_present`	All 8 required columns must exist; the pipeline raises `ValueError` if Yahoo's schema ever changes.
2	`duplicate_ticker_date`	Each `(ticker, date)` pair must be unique to satisfy the database `UNIQUE` constraint and prevent double-counting.
3	`null_required_values`	Rows missing any required price/volume/date are dropped — they would corrupt SQL aggregations and ML features.
4	`negative_or_zero_prices`	Stock prices must be strictly positive; non-positive values indicate data corruption.
5	`price_relationship_invariants`	Within a trading day, `low ≤ open, close ≤ high` and `low ≤ high`; any violation is physically impossible.

Soft checks (flag and report only)

#	Rule	Rationale
6	`zero_volume_days`	Zero-volume days can indicate trading halts or data gaps; flagged but kept (halts are valid market events).
7	`extreme_daily_returns_gt_50pct`	Single-day moves above 50% almost always indicate an unadjusted split or data error rather than a real return.
8	`date_gap_above_5_calendar_days`	Calendar gaps > 5 days within a ticker may indicate missing trading-day records (excludes normal weekend/holiday gaps).

Result on the live dataset

On the current Yahoo Finance dataset (6,270 rows across AAPL, MSFT, TSLA), all eight rules ran and reported **0 row-level violations** — the source proved to be already clean for these large-cap stocks. The cleaning pipeline is still essential because:

- We cannot guarantee future data will be clean — Yahoo's schema or data quality may change without notice.

- The pipeline normalizes column names, date strings, and numeric types so downstream stages have a stable contract.
- The validation report acts as a regression check: if the API ever returns dirty data, the validation CSV will surface it immediately rather than letting it propagate silently.

Verification on synthetic dirty data

To prove the cleaning logic actually works, `src/data/test_cleaning_on_dirty_fixture.py` runs the production cleaner against an 8-row synthetic CSV containing one duplicate row, one row with a `null` close, one row with negative prices, and two rows that violate price-relationship invariants. The test passes when the cleaner removes exactly 5 of 8 rows, with each removal attributed to the correct rule:

```
Per-check breakdown:
  - schema_required_columns_present     failed=0  action=raise_if_missing
  - duplicate_ticker_date               failed=1  action=drop_duplicates
  - null_required_values                failed=1  action=drop_rows
  - negative_or_zero_prices             failed=1  action=drop_rows
  - price_relationship_invariants       failed=2  action=drop_rows

PASSED: cleaning pipeline correctly removed all dirty rows.
```

The cleaned dataset is then saved as a single consolidated CSV (`data/processed/cleaned_stock_prices.csv`) so later project stages reuse the same data source without re-running the cleaning process.

5. Exploratory Analysis and Visualization

Exploratory data analysis focused on understanding price movement, trading activity, return distributions, and dataset coverage for the three selected stocks. Eleven visualizations were generated, organized into three categories:

Price and Volume Trends (5 figures)

#	Visualization	Purpose
1	Closing price trends	Raw price trajectory for each stock
2	Adjusted closing price trends	Price trajectory accounting for splits/dividends
3	Indexed adjusted close growth	Relative growth comparison from a common base (100)
4	Trading volume trends	Raw daily trading volume
5	30-day rolling average volume	Smoothed volume to reduce daily noise

Summary and Comparison (3 figures)

#	Visualization	Purpose

6	Row count per stock	Data completeness check
7	Average closing price by stock	Quick cross-stock price comparison
8	Yearly average closing price trend	Long-term price evolution (complete years only)

Returns and Volatility Analysis (3 figures)

#	Visualization	Purpose
9	Daily returns distribution	Histogram + KDE showing return shape, mean, and std per stock
10	Daily returns correlation heatmap	Measures how closely the three stocks move together day-to-day
11	Monthly returns boxplot	Compares return spread and outliers across stocks

Key Observations

- **Microsoft** has the highest average close price in the cleaned dataset, reflecting its steady long-term growth trajectory.
- **Tesla** has the highest average trading volume, which aligns with its higher retail investor activity and stock volatility.
- **Row counts** are identical across the three stocks, confirming the dataset is perfectly balanced for comparative analysis and later modeling.
- **Daily returns**: Tesla has the widest distribution (highest standard deviation), confirming it is the most volatile of the three. All three distributions are approximately centered around zero with slight positive skew.
- **Correlation**: Apple and Microsoft show a moderate-to-strong positive correlation in daily returns, which makes sense since they are both large-cap technology companies. Tesla is less correlated with the other two.
- **Monthly returns boxplot**: Tesla has the widest interquartile range and the most extreme outliers, further confirming its higher volatility.

Visualization Design Decisions

1. **Indexed growth plot**: Raw price levels are not effective for comparing growth across stocks with very different price ranges. The indexed adjusted close normalizes each stock to a common base value of 100 at the start date, making relative growth directly comparable.

2. **Complete-year filtering**: The yearly average trend excludes the incomplete current year so that yearly comparisons are not distorted by partial data. Years with fewer than 200 trading days are filtered out.

3. **Returns distribution with statistics**: Each stock's daily return histogram includes an overlaid KDE curve and annotated mean (μ) and standard deviation (σ), which quantifies the risk profile beyond what a simple trend line can show.

All figures are saved at 150 DPI for report-quality resolution, with formatted axis labels (dollar signs for prices, millions notation for volumes), consistent color coding, and descriptive legends.

Descriptive Statistics

A full descriptive statistics table is exported to `outputs/metrics/descriptive_statistics.csv`, including mean, median, standard deviation, skewness, and kurtosis of daily returns for each stock.

6. Feature Engineering and Machine Learning

Feature Engineering

To prepare the cleaned stock data for machine learning, 11 numerical features were engineered from the raw price and volume fields. All features were computed within each ticker group to avoid data leakage across stocks. The feature engineering pipeline reads the cleaned dataset and outputs a new `featured_stock_prices.csv` with the original columns preserved alongside the new features.

Engineered features:

Feature	Description	Rationale
`daily_return`	Percent change in closing price from the previous day	Captures day-to-day price momentum
`lag_1` through `lag_5`	Closing prices from 1 to 5 days prior	Provides historical price context for time-series prediction
`ma_5`, `ma_10`, `ma_20`	5-, 10-, and 20-day simple moving averages of close	Smooths short-term noise to reveal medium-term trends
`volatility_10`	Rolling 10-day standard deviation of daily returns	Quantifies recent price instability
`volume_change`	Percent change in trading volume from the previous day	Captures shifts in market activity

Target variable: `target_next_day_up` — a binary label (1 if the next trading day's closing price is higher than today's, 0 otherwise). This frames the problem as binary classification: predicting the direction of the next-day price movement.

Rows with NaN values (caused by lag/rolling-window computations at the start of each ticker's history) were dropped, leaving 6,213 rows available for modeling.

Feature Correlation Analysis

A feature correlation heatmap revealed important patterns in the engineered features:

- **Lag features** (`lag_1` through `lag_5`) are extremely highly correlated with each other (>0.99), which is expected since consecutive closing prices change incrementally. This multicollinearity does not prevent tree-based models from training, but it means these features carry largely

redundant information.

- **Moving averages** (`ma_5`, `ma_10`, `ma_20`) are also highly correlated with each other and with the lag features, for the same reason — they are all smoothed versions of the same underlying price series.
- **`daily_return`**, **`volatility_10`**, and **`volume_change`** are largely independent of the lag/MA cluster, making them the most informative features for distinguishing between up and down days.

Model Selection

Three classification models were trained, chosen to represent increasing levels of complexity:

Model	Why Selected
Logistic Regression	Linear baseline; fast to train, interpretable coefficients, assumes linearly separable classes
Decision Tree	Non-linear; captures threshold-based decision rules, easy to interpret
Random Forest	Ensemble of decision trees; reduces overfitting through bagging and feature randomness

All models were trained on the same feature set: the 11 engineered features plus one-hot-encoded ticker dummy variables (to control for stock-specific intercept differences).

Train/Test Split

A **chronological split** was used instead of random cross-validation. For each ticker, the first 80% of trading days (by date) formed the training set and the last 20% formed the test set. This prevents future-data leakage: the model never sees tomorrow's data during training, which mirrors how any real prediction system would operate. Random splitting would allow the model to "peek" at future prices through nearby lag features, artificially inflating accuracy.

Naive Baseline

Before evaluating the trained models, a **naive majority-class baseline** was established. This baseline predicts the most common class in the training set (up or down) for every test sample. Any useful model must outperform this floor; otherwise, it is no better than always guessing the majority label.

7. Results and Discussion

Model Performance

Model	Accuracy	Precision	Recall	F1	AUC
Naive Baseline	0.5205	0.5205	1.0000	0.6846	—
Random Forest	0.5141	0.5311	0.5664	0.5482	0.5006

Decision Tree	0.5133	0.5180	0.9306	0.6656	0.5048
Logistic Regression	0.4996	0.5146	0.6806	0.5860	0.4909

(Values from the latest pipeline run; full results in `outputs/metrics/model_comparison.csv` and `outputs/metrics/roc_auc_scores.csv`.)

Interpretation

The results show that none of the models meaningfully outperform the naive baseline. This is a significant finding, not a failure:

1. **The Decision Tree degenerates toward the baseline.** Its 93% recall and 52% precision indicate that it predicts "up" for nearly every sample. This is essentially the same strategy as the naive baseline — always guess the majority class — confirming that it has not learned a useful decision boundary.

2. **Logistic Regression and Random Forest perform near chance (50%).** Neither model achieves accuracy above the naive baseline. The ROC curves confirm this: AUC scores (0.49–0.50 across all three models) hover essentially at random, meaning the predicted probabilities carry essentially no discriminative signal.

3. **This aligns with the Efficient Market Hypothesis (EMH).** The EMH states that publicly available information is already reflected in stock prices, making short-term price movements unpredictable from historical data alone. Our features — lagged prices, moving averages, volatility, and volume changes — are all publicly available and widely monitored. If these features contained easily exploitable predictive power, market participants would already have traded on them, eliminating the advantage. The ~50% accuracy is the expected outcome under semi-strong EMH.

4. **Feature importance from Random Forest** ranks `daily_return`, `volume_change`, and `volatility_10` as the most informative features. The lag features and moving averages — despite being highly correlated with each other — contribute relatively little discriminative power. This suggests that raw price levels (lags, MAs) are poor predictors of direction, while measures of *change* and *instability* (returns, volatility, volume shifts) carry slightly more information, though still not enough to overcome market noise.

ROC Curve Analysis

The ROC curves for all three models hug the diagonal (random classifier line), with AUC values near 0.50. This confirms the accuracy results: the models' predicted probabilities do not meaningfully separate up-days from down-days. If the models had learned useful patterns, the ROC curves would bow toward the upper-left corner with AUC values significantly above 0.50.

Confusion Matrix Analysis

The confusion matrices reveal different failure modes:

- **Decision Tree**: heavy bias toward predicting "up" (very few "down" predictions), achieving high recall for the up class but near-zero recall for the down class

- **Logistic Regression**: more balanced predictions but still near-chance accuracy for both classes
- **Random Forest**: similar to logistic regression, with slightly more conservative up predictions

Connecting to the Research Question

The research question asked whether historical stock market data can predict short-term stock price movements. The answer from this project is: **with the features and models used, no.** Historical price and volume data alone do not provide sufficient signal to predict next-day price direction above a naive baseline.

This is a well-documented result in quantitative finance and validates that the project's methodology is sound — the pipeline is working correctly, and the models are being evaluated fairly. The finding itself is valuable because it demonstrates understanding of why stock prediction is fundamentally difficult and why more sophisticated approaches (sentiment analysis, alternative data, deep learning on longer sequences) are active areas of research.

8. Limitations and Future Work

Data Limitations

The pipeline only uses market price and volume fields from Yahoo Finance. This makes the dataset manageable and easy to explain, but it also means that potentially useful external information — such as earnings announcements, macroeconomic indicators, or news sentiment — is not included. Financial markets are inherently noisy and influenced by many factors that may not appear in historical price data alone.

Technical Limitations

- The database stores data in SQLite, which is appropriate for a course project but would not scale well to a production system that needs concurrent writes or real-time data ingestion.
- The current pipeline downloads data in batch mode and does not support incremental updates.
- Only three stocks were analyzed. A broader universe of stocks could reveal sector-level patterns not visible in this small sample.

Model Limitations

- The chronological split evaluates each model on a single test period. Time-series cross-validation (e.g., walk-forward validation) would provide a more robust estimate of model performance across different market regimes.
- No hyperparameter tuning was performed beyond the initial configuration. Grid search or randomized search could marginally improve model performance.
- Only classification models were tested. Regression approaches (predicting return magnitude rather than direction) or sequence models (LSTM, Transformer) could capture temporal dependencies that tree-based models miss.

Future Work

- Additional technical indicators (RSI, MACD, Bollinger Bands)
- External financial indicators or sentiment-based variables
- Sequence models (LSTM, Transformer) that can learn longer temporal patterns
- Incremental data updates instead of full re-downloads
- Walk-forward cross-validation for more robust evaluation
- A more scalable database backend (e.g., PostgreSQL)

9. Individual Contributions

Ryan Amir

- Data acquisition pipeline (download from Yahoo Finance via `yfinance`)
- Data cleaning and validation pipeline (Pandas)
- Relational database design and loading (SQLite), including sector metadata
- SQL report queries: 11 queries including correlated subqueries, CASE expressions, and sector joins
- Exploratory data analysis: 11 visualizations (price trends, returns distributions, correlation heatmap, monthly volatility)
- Descriptive statistics export (mean, median, std, skewness, kurtosis)
- EDA summary statistics and markdown export
- Master pipeline script (`run_pipeline.py`)
- Unified project pipeline (`run_all.py`)
- Project documentation (README, data dictionary, oral exam notes, final report)

Gian Cases

- Feature engineering (daily returns, lag features, moving averages, volatility, volume change)
- Target variable design (next-day price direction as binary classification)
- Naive baseline evaluation (majority-class predictor)
- Model training (Logistic Regression, Decision Tree, Random Forest)
- Model evaluation (accuracy, precision, recall, F1, confusion matrices, classification reports, ROC curves with AUC)
- ML-focused visualizations (feature importance, feature correlation heatmap, model comparison, ROC curves)
- ML pipeline script (`run_ml_pipeline.py`)
- Results interpretation and connection to Efficient Market Hypothesis

References

- Yahoo Finance: https://finance.yahoo.com
- yfinance Python library: https://pypi.org/project/yfinance/
- Pandas documentation: https://pandas.pydata.org/docs/
- Matplotlib documentation: https://matplotlib.org/stable/
- SQLite documentation: https://www.sqlite.org/docs.html

- scikit-learn documentation: https://scikit-learn.org/stable/
- Fama, E. (1970). "Efficient Capital Markets: A Review of Theory and Empirical Work." *The Journal of Finance*, 25(2), 383-417.